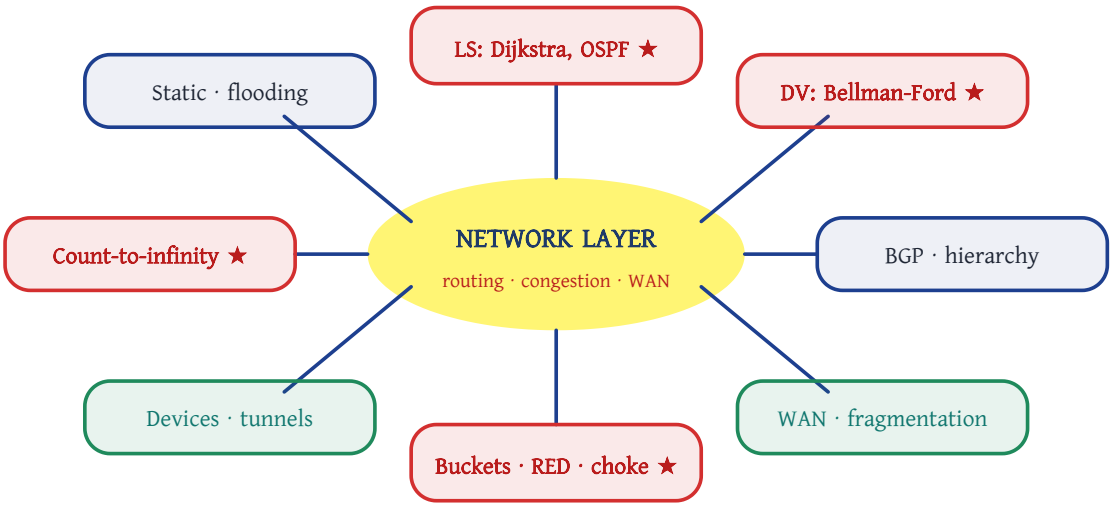
# Routing, Congestion Control & WAN

Dijkstra · Bellman-Ford · BGP · buckets · devices · X.25/FR/ATM/MPLS · fragmentation: exam-ready in 12 sheets




↑ the whole lesson on one page: red = high-yield, green = applied

## Syllabus coverage

"Computer Networking and Security: ... **Routing algorithm**; **Congestion control**; **Internetworking, WAN**; **Fragmentation**; ...". Neighbouring notes: layers & LANs (IEEE 802), IPv4/IPv6 header and addressing, ARP (network layer notes), TCP's congestion window (transport notes).

**High-yield list**

① DV → Bellman-Ford
② LS → Dijkstra
③ RIP: 16 hops = ∞
④ split horizon
⑤ $S = C/(M - \rho)$
⑥ offset ÷ 8 bytes
⑦ ATM cell 53 B

**Legend:** **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

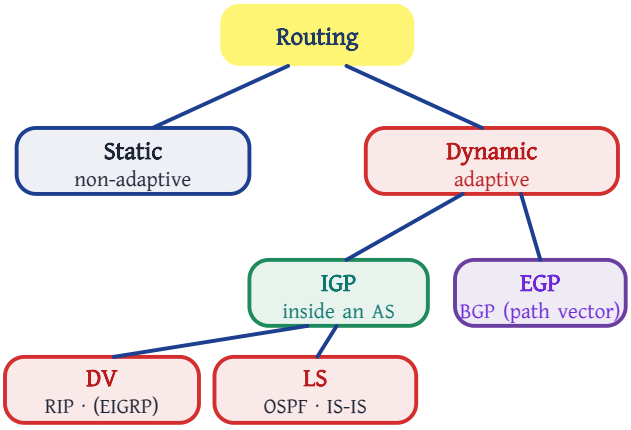
## Contents


✓ Routing basics, flooding ..... p. 2
✓ Link state, Dijkstra, OSPF, hierarchy ... p. 3
✓ Distance vector, RIP, count-to-∞ ... p. 4
✓ BGP, broadcast, congestion ..... p. 5
✓ Choke packets, RED, shaping ..... p. 6
✓ Leaky & token bucket, devices ... p. 7
✓ Tunnelling, switching ..... p. 8
✓ WAN: leased lines, X.25, FR, ATM, MPLS ... p. 9
✓ IPv4 fragmentation ..... p. 10
✓ Night-before sheet ..... p. 11
✓ Practice MCQs + written ..... p. 12

## 1 Routing: the Big Picture

**Definition:** A **routing algorithm** is the network-layer (layer 3) software that decides on which output line an incoming packet should be sent. **Routing** = building the routing table (control plane); **forwarding** = looking up each packet in that table and sending it out (data plane).

**Optimality principle:** If J is on the optimal path from I to K, the optimal path from J to K follows the same route. So all optimal routes to one destination form a tree: the **sink tree** (no loops).




the family tree of routing

| | Static (non-adaptive) | Dynamic (adaptive) |
|---|---|---|
| Routes set by | administrator, by hand | protocol exchanges between routers |
| On failure · overhead · use | no reaction · none · small/stub networks | re-routes itself · bandwidth, CPU · large networks |

**Metrics**: hop count (RIP) · bandwidth · delay · load · reliability · MTU · cost (OSPF) · AS path / policy (BGP). **Lower metric = better** route.

**Memory trick:** "**RIP counts hops, OSPF counts cost, BGP counts ASes** (and obeys policy)".

**Exam trap:** **Metric** compares routes **within one protocol**; **administrative distance** (AD, trustworthiness) chooses **between protocols**, lower wins. Cisco AD: connected 0 · static **1** · eBGP 20 · EIGRP 90 · OSPF 110 · IS-IS 115 · RIP 120 · iBGP 200. So an OSPF route beats a RIP route whatever the hop count.

## 2 Flooding

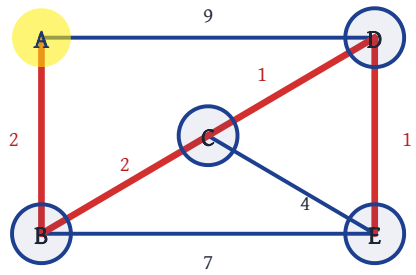
A static algorithm: send every packet out on **every line except the one it arrived on.** Robust, and always finds the shortest path (every path is tried), but floods the network with duplicates. Damp it with a **hop counter** (start at the diameter), **sequence numbers** (drop copies already seen) or **selective flooding**. Used to distribute link-state packets.

# 3 Link State Routing & Dijkstra

**Idea:** Every router learns the **complete topology map** and runs **Dijkstra's algorithm** (SPF) on it itself: "routing by map".

1. **Discover neighbours** (HELLO) · **measure the cost** to each · **build a link state packet** (LSP/LSA: ID, sequence no., age, neighbours + costs)
2. **Flood** the LSP to **all** routers · **run Dijkstra** on the full map → shortest-path tree → routing table

## Worked example: shortest paths from A




red = final shortest-path tree

| Step | Permanent set | B | C | D | E |
|---|---|---|---|---|---|
| 0 | {A} | 2,A | ∞ | 9,A | ∞ |
| 1 | {A,B} | ✓ | 4,B | 9,A | 9,B |
| 2 | {A,B,C} | ✓ | ✓ | 5,C | 8,C |
| 3 | {A,B,C,D} | ✓ | ✓ | ✓ | 6,D |

"4,B" = distance 4 via previous node B. Each step makes the **smallest tentative** node permanent and relaxes its neighbours. Every route from A leaves via next hop **B**; A→E = A-B-C-D-E, cost 6 (not the direct-looking A-D-E = 10).

**OSPF facts:**

RFC 2328 (v2), v3 for IPv6

runs **directly on IP**: protocol **89**

cost = $10^8$ / bandwidth (100 Mbps → 1)

**areas**; backbone = **area 0**; ABR, ASBR, DR/BDR

224.0.0.5 all routers · 224.0.0.6 DRs

Hello 10 s, dead 40 s · AD 110

**Memory trick:** OSPF in numbers: "**89 · 110 · area 0 · 5/6 · 10/40**" = protocol, AD, backbone, multicast ends, hello/dead.

**IS-IS**: OSI link-state twin, runs over layer 2, not IP. Dijkstra: $O(N^2)$.

**Exam trap:** Dijkstra fails with **negative** edge costs; Bellman-Ford handles them (but not negative cycles).

## 4 Hierarchical Routing

Routers are grouped into **regions** (areas, ASes); a router keeps every router of its own region but only **one entry per other region**. Smaller tables, slightly longer paths. Optimum: **ln N** levels, **e·ln N** entries.

**Worked example ✍️:** 720 routers. Flat: **720** entries. 24 regions × 30 routers: 30 + 23 = **53**. 8 clusters × 9 regions × 10 routers: 10 + 8 + 7 = **25**.

## 5 Distance Vector Routing & Bellman-Ford

**Idea:** Each router keeps the best known **distance** (and line) to every destination and swaps this vector **only with its neighbours**, periodically. "Routing by rumour"; ARPANET until 1979, then RIP.

**Bellman-Ford equation:** $D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$ over all neighbours v of x

**Worked example ✍️:** Router X: links X–A = 2, X–B = 5, X–C = 1. Received vectors:

| from | to Y | to Z |
|---|---|---|
| A | 7 | 3 |
| B | 2 | 9 |
| C | 8 | 6 |

Y: min(2+7, **5+2**, 1+8) = **7 via B**

Z: min(**2+3**, 5+9, 1+6) = **5 via A**

**RIP facts:**

metric = **hop count**; max **15**, **16 = ∞**

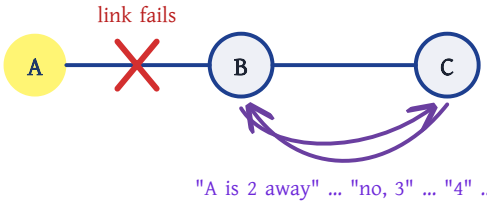
full table every **30 s** · UDP port **520**

invalid 180 s · hold-down 180 s · flush 240 s

v1 classful, broadcast · v2 classless, multicast **224.0.0.9**, auth

RIPng (IPv6): UDP 521 · AD 120

## 6 Count-to-Infinity & Its Fixes




| exchange | B's dist to A | C's dist to A |
|---|---|---|
| before failure | 1 | 2 |
| 1 | 3 (via C!) | 2 |
| 2 · 3 · 4 | 3 · 5 · 5 | 4 · 4 · 6 |
| finally | **16 = ∞** | **16 = ∞** |

C still advertises "A at 2" (a route through B itself) and B believes it. **Good news travels fast, bad news travels slowly.**

| Fix | How it works |
|---|---|
| **Split horizon · poison reverse** | never advertise a route back on the interface it came from · or send it back with metric 16 (∞) |
| **Route poisoning · triggered updates** | a failed route is sent at once with metric 16, not after 30 s |
| **Hold-down timer** | after a route goes down, ignore worse news about it for 180 s |

**Exam trap:** Split horizon cures the two-router loop but **not** loops of three or more routers; RIP's small ∞ (16) only caps the count.

| | Distance vector | Link state |
|---|---|---|
| Knows · algorithm | neighbours' distances · **Bellman-Ford** | whole topology · **Dijkstra** |
| Sends | whole table, to neighbours, periodically | own links (LSA), to all routers, on change |
| Convergence · CPU | slow, count-to-∞ · low, small nets | fast, loop-free · high, large nets |
| Examples | RIP, IGRP; EIGRP = "advanced DV / hybrid" (DUAL) | OSPF, IS-IS |

## 7 BGP: Routing Between Autonomous Systems

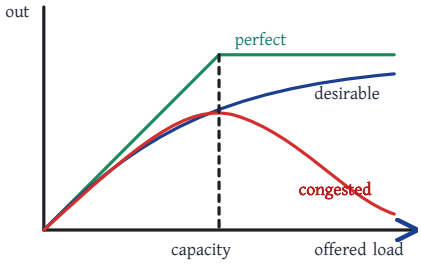
- **Path vector** protocol; BGP-4 is the Internet's inter-domain (EGP) protocol
- runs over **TCP port 179**; eBGP between ASes, iBGP inside one
- a route carries its full **AS_PATH**; a router rejects a path that holds its own AS → **no loops**
- chooses by **policy** (LOCAL_PREF, AS_PATH length, MED), not by cost · messages: OPEN, UPDATE, KEEPALIVE, NOTIFICATION

## 8 Broadcast & Multicast Routing

**Broadcast**: separate copies (wasteful) · flooding · multidestination · **spanning tree** (optimal) · **reverse path forwarding** (RPF): forward a copy only if it arrived on the line this router uses to reach the source. **Multicast** (224.0.0.0/4, IGMP): source-based trees (DVMRP, PIM-DM) or shared core-based trees (CBT, PIM-SM).

## 9 Congestion & Its Control

**Definition:** **Congestion**: too many packets in (part of) the network, so delay rises and throughput falls. **Congestion control** is global (the whole network); **flow control** is point-to-point (fast sender, slow receiver).




packets delivered vs offered

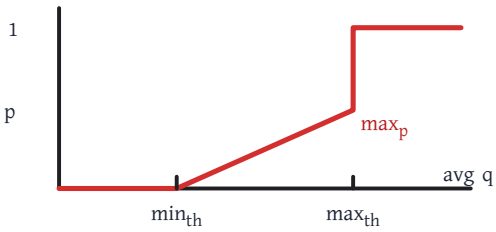
**Causes:** bursty input converging on one line, slow routers, too few buffers, low-bandwidth lines.

Infinite buffers make it **worse**: packets queue so long they time out and are retransmitted, adding load. Past capacity output can fall towards 0: **congestion collapse**.

| | Open loop (prevention) | Closed loop (feedback, reaction) |
|---|---|---|
| Idea | good design so congestion never starts | monitor, signal, then adjust |
| Tools | provisioning, **admission control**, traffic shaping (buckets), retransmission/ACK/discard policies | **choke packets**, back-pressure, ECN, RED, TCP's AIMD window (transport notes) |

- **Admission control**: in virtual-circuit networks, refuse a new VC that would congest the network (the telephone "busy" tone).
- **Choke packet**: a congested router sends a packet **back to the source**, which cuts its rate (e.g. by half). **Hop-by-hop back-pressure**: the choke takes effect at every router it passes, so relief is immediate. (ICMP Source Quench is now deprecated.)
- **ECN**: the router **marks** 2 bits in the IP header instead of dropping; the receiver echoes it (TCP ECE flag), the sender slows down.
- **Load shedding**: drop packets. **Wine** (old is better: file transfer, drop the new) vs **milk** (new is better: real-time media, drop the old).

**RED** (Random Early Detection, Floyd & Jacobson 1993): the router tracks the **average** queue length. Below $min_{th}$: no drop; between thresholds: drop with probability rising to $max_p$; above $max_{th}$: drop all. Dropping **early and at random** slows a few TCP senders and avoids tail-drop's **global synchronisation**.

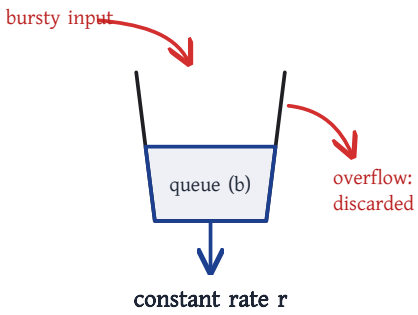



RED drop probability

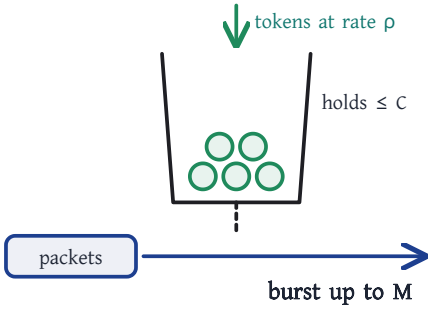
**Exam trap:** Congestion control ≠ flow control: a fast PC sending to a slow printer on an idle link is a flow-control problem; many hosts on one overloaded link is congestion.

## 10 Traffic Shaping: Leaky & Token Bucket

**Traffic shaping:** regulating the **rate and burstiness** of the traffic a host sends, as agreed with the network (open loop). **Policing** = the network checking it and dropping or marking the excess.




leaky bucket: smooth output




token bucket: bursts allowed

| | Leaky bucket | Token bucket |
|---|---|---|
| Output | **constant** rate r, however bursty the input | average rate ρ, bursts up to M allowed |
| Idle host · when full | gains nothing · discards **packets** | saves up to C tokens · discards **tokens** |
| Max data in time t | r·t | **C + ρt** (and ≤ M·t) |

**Burst length:** Full bucket C bytes, token rate ρ, maximum output rate M. During a burst of S seconds: C + ρS = MS ⇒ **S = C / (M − ρ)**

**Example 1 ✍️:** C = 250 KB, M = 25 MB/s, ρ = 2 MB/s: S = 250 KB / 23 MB/s ≈ **11 ms** at full speed, then 2 MB/s.

**Example 2 ✍️:** 1 MB burst (25 MB/s for 40 ms) into a leaky bucket, output 2 MB/s: output lasts **500 ms**; bucket must hold ≈ 0.92 MB.

**Memory trick:** **Leaky** = a dripping tap: always the same drip. **Token** = a savings account: save while idle, spend in a burst, never more than the balance C.

## 11 Internetworking & Its Devices

**Definition:** **Internetworking** = joining dissimilar networks (addressing, packet size, service, QoS) so they act as one; layer-3 routers running IP glue the Internet together.

| Device | Layer | Works on | Collision domains | Broadcast domains |
|---|---|---|---|---|
| Repeater | 1 | bits: regenerates the signal | 1 (extends it) | 1 |
| Hub | 1 | bits: multiport repeater | 1 for all ports | 1 |
| Bridge | 2 | frames: MAC filtering | 1 per port | 1 |
| Switch | 2 | frames: multiport bridge | **1 per port** | 1 (per VLAN) |
| Router | 3 | packets: IP address | 1 per port | **1 per port** |
| Gateway | 4–7 | protocol conversion | – | – |

- **Bridge: backward learning** of MAC → port from source addresses; floods unknown destinations; STP (802.1D) removes loops.

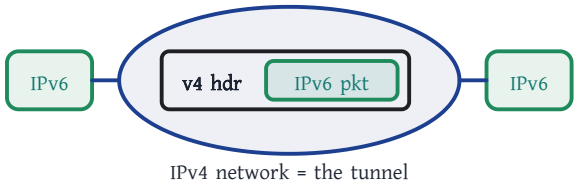
- **Switch**: store-and-forward (checks CRC) or cut-through (lower latency); full duplex, no collisions; a layer-3 switch also routes.
- **Gateway**: converts between dissimilar protocols (e-mail, VoIP); the TCP/IP "default gateway" is just the router.

> **Worked example ✍️:** A router with 2 interfaces: one to a 24-port switch with 20 PCs, one to a hub with 6 PCs. Collision domains: 20 + 1 (switch–router link) + 1 (whole hub side) = **22**. Broadcast domains: **2** (one per router interface).

> **Exam trap:** A **switch** breaks collision domains but **not** broadcast domains; only a **router** (or VLANs) splits broadcast domains. A hub is a layer-1 device even though it has many ports.

### Tunnelling

To cross a network of a **different type**, the whole packet travels as the **payload** of the foreign network's packet between two multiprotocol routers. Examples: IPv6-in-IPv4 (protocol 41), GRE (protocol 47), IPsec VPN.




## 12 Circuit vs Packet Switching

| | Circuit | Datagram | Virtual circuit |
|---|---|---|---|
| Idea | dedicated path set up first | each packet routed alone | path fixed at setup, links shared |
| Packet carries | – | full address | short VC number |
| Order · router fails | in order · call lost | may be out of order · only its packets lost | in order · all VCs through it end |
| QoS, congestion | easy (reserved) | difficult | easy if resources reserved |
| Examples | PSTN, ISDN | **IP** | **X.25, Frame Relay, ATM, MPLS** |

> **Delay formulas:** message x bits, k hops, bit rate b, propagation d per hop, setup s, packet p bits (headers ignored): circuit $\mathbf{T = s + x/b + k{\cdot}d}$ · packet $\mathbf{T = x/b + (k - 1){\cdot}p/b + k{\cdot}d}$. Packet switching wins when $s > (k - 1){\cdot}p/b$.
> **Example ✍️** $x = 10^6$ bits, b = 1 Mbps, k = 4, d = 5 ms, s = 200 ms, p = 10 000 bits: circuit 0.2 + 1 + 0.02 = **1.22 s**; packet 1 + 3 × 0.01 + 0.02 = **1.05 s**.

> **Exam trap:** A virtual circuit is **packet** switching (links are shared), not circuit switching; it only fixes the route at setup.

# 13 WAN Technologies

A **WAN** spans cities, countries, continents, usually over a carrier's lines: leased lines, circuit-switched ISDN, packet-switched X.25 / Frame Relay / ATM, MPLS, Internet VPN (IPsec), satellite/cellular backup.

| Leased line | Rate | Channels |
|---|---|---|
| T1 (USA) | **1.544 Mbps** | 24 × 64 k |
| E1 (Europe, BD) | **2.048 Mbps** | 32 (30 usable) |
| T3 / E3 | 44.736 / 34.368 | 672 / 480 |
| OC-3 = STM-1 | 155.52 Mbps | SONET/SDH |
| OC-12 = STM-4 | 622.08 Mbps | |

**ISDN**: BRI = 2B + D = 2 × 64 + 16 = **144 kbps**; PRI = 23B + D (T1) or 30B + D (E1).

T1 = 24 × 64 kbps + 8 kbps framing = 1.544 Mbps. OC-1 = 51.84 Mbps; OC-n = n × 51.84.

| | X.25 | Frame Relay | ATM |
|---|---|---|---|
| Era · body | 1976, CCITT/ITU-T | early 1990s | 1990s, B-ISDN core |
| Layers | 3 (X.21, LAPB, PLP) | 2 only (LAPF) | physical, ATM, AAL |
| Unit | packet (variable) | frame (variable) | **53-byte cell** |
| Error control | **hop-by-hop** (noisy lines) | none: bad frames dropped | HEC on header only |
| VC identifier | LCN | **DLCI** (10 bits) | **VPI/VCI** |
| Speed | ≤ 64 kbps (later 2 Mbps) | 56 kbps – 45 Mbps | 155.52 / 622.08 Mbps |
| Extra | PAD for terminals | CIR, DE bit, FECN/BECN | CBR, VBR, ABR, UBR |




ATM cell




MPLS label entry

**ATM**: 48 bytes was a compromise (USA wanted 64, Europe 32). UNI header: GFC 4, VPI 8, VCI 16, PT 3, CLP 1, HEC 8 bits. The ATM network layer and AAL are in their own notes.

**MPLS** ("layer 2.5", RFC 3031): the ingress **LER** classifies a packet into a FEC and **pushes** a label; core **LSRs swap** labels (exact match, fast); the egress **pops** it. The path is an **LSP**. Carriers use it for traffic engineering, QoS and L3 VPNs; it carries any protocol.

> Memory trick: **X.25 checks every hop, Frame Relay checks none, ATM chops everything into 53.**
> Frame Relay = "X.25 with the error control thrown out, because fibre is clean".

> Exam trap: ATM's HEC protects only the **5-byte header**, not the payload. Frame Relay does **detect** errors (FCS) but never **corrects** them: it discards the frame.

## 14 IPv4 Fragmentation

**Why:** A datagram bigger than the next link's **MTU** (largest frame payload) is split into **fragments**, each a full IP datagram with its own header. IPv4: the source **or any router** fragments; **reassembly only at the final destination.**

| Header field (bits) | Role |
|---|---|
| Identification (16) | same in all fragments |
| DF (1) · MF (1) | don't fragment · more follow |
| Fragment offset (**13**) | in **8-byte** units |
| Total length (16) | header + data ≤ 65 535 |

| Link | MTU (B) |
|---|---|
| Ethernet · PPP | **1500** |
| 802.11 · FDDI | 2304 · 4352 |
| X.25 (typical) | 576 |
| IPv6 minimum | 1280 |

**Worked example 1 ✍️:** Total length 4000 B (20-byte header), next link MTU 1500.

1. Data = 4000 − 20 = 3980 B. Max data per fragment = 1500 − 20 = 1480 (a multiple of 8 ✓)
2. Split 3980 = 1480 + 1480 + 1020

| Frag | Data bytes | Total length | Offset | MF |
|---|---|---|---|---|
| 1 | 0 – 1479 | 1500 | 0 | 1 |
| 2 | 1480 – 2959 | 1500 | 1480/8 = 185 | 1 |
| 3 | 2960 – 3979 | 1040 | **2960/8 = 370** | 0 |

**Example 2: the 8-byte trap ✍️:** MTU 1000, datagram 2400 B. 1000 − 20 = 980, not a multiple of 8 → use **976**. Data 2380 = 976 + 976 + 428; offsets **0, 122, 244**.

**Example 3: read a fragment ✍️:** offset 100, HLEN 5, total length 100 → first data byte = 100 × 8 = **800**, data = 80 B, last byte **879**.

- MF = 1, offset = 0 → **first** fragment · MF = 0, offset ≠ 0 → **last** · MF = 0, offset = 0 → not fragmented
- One fragment lost → the **whole datagram** is discarded (reassembly timer)
- **Path MTU discovery**: send with DF = 1; a router that cannot forward drops it and returns ICMP type 3 code 4 "fragmentation needed" with the next-hop MTU
- **IPv6**: routers **never** fragment; only the source (Fragment header, 8 B, next header 44); too big → ICMPv6 "Packet Too Big"

**Memory trick:** "**Offset × 8 = byte position**": $2^{13}$ offsets × 8 bytes = $2^{16}$, which covers the 65 535-byte datagram.

# Night-Before-the-Exam Sheet

**Fact sheet:**

DV: $D_x(y) = \min\{c(x,v) + D_v(y)\}$

LS: flood LSAs, then Dijkstra $O(N^2)$

RIP: hops, 15 max, 16 = ∞, 30 s, UDP 520

OSPF: IP proto 89, area 0, AD 110

BGP: path vector, TCP 179, AS_PATH

AD: static 1, eBGP 20, OSPF 110, RIP 120

Hierarchy: ln N levels, e·ln N entries

Token: $S = C/(M - \rho) \cdot \max = C + \rho t$

Circuit: s + x/b + kd

Packet: x/b + (k − 1)p/b + kd

ATM 53 = 5 + 48 · VPI/VCI

T1 1.544 · E1 2.048 · STM-1 155.52

MPLS: label 20 · TC 3 · S 1 · TTL 8

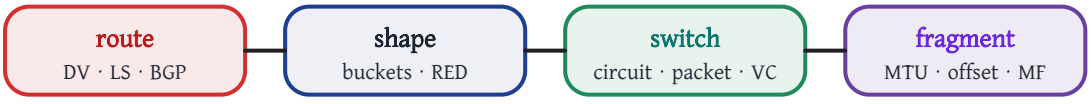
Offset = byte pos / 8 (13 bits)

## Top 14 one-liners for MCQs

1. Router = layer 3, bridge/switch = 2, hub/repeater = 1
2. Distance vector uses Bellman-Ford
3. Link state uses Dijkstra
4. BGP is the inter-AS (EGP) protocol
5. Split horizon fights count-to-infinity
6. Optimal routes to one node form a sink tree
7. Flooding tries every path: robust, many duplicates
8. Leaky bucket → constant output rate
9. Token bucket → bursts allowed, keeps tokens
10. RED drops early, at random, on avg queue
11. X.25 = hop-by-hop error control
12. Frame Relay uses DLCI; ATM uses VPI/VCI
13. IPv4 reassembly only at the destination
14. IPv6 routers never fragment

**Top traps:** metric ≠ administrative distance · split horizon fails for 3+ router loops · switch splits collision, not broadcast, domains · virtual circuit = packet switching · leaky bucket drops packets, token bucket drops tokens · offset counts 8-byte units, and data per fragment must be a multiple of 8 · ATM HEC covers the header only

**All tricks together:** RIP counts hops, OSPF counts cost, BGP counts ASes · OSPF "89 · 110 · area 0 · 5/6 · 10/40" · RPF = came in on my road home? · leaky = dripping tap, token = savings account · X.25 checks every hop, FR none, ATM chops into 53 · offset × 8 = byte position




the four questions every network-layer paper asks

## ✓ Practice MCQs (exam pattern, 10 marks)

**1.** Distance vector routing is based on:
(a) Dijkstra (b) Prim
(c) Bellman-Ford (d) Kruskal

**2.** In RIP, a hop count of 16 means:
(a) unreachable (b) best route
(c) 16 routers (d) default route

**3.** OSPF is carried directly in IP with protocol number:
(a) 6 (b) 17
(c) 520 (d) 89

**4.** Which protocol routes between autonomous systems?
(a) BGP (b) RIP
(c) OSPF (d) IS-IS

**5.** The leaky bucket algorithm mainly provides:
(a) shortest paths (b) constant output rate
(c) error detection (d) address resolution

**6.** Token bucket: C = 250 KB, ρ = 2 MB/s, M = 25 MB/s. Burst length ≈
(a) 125 ms (b) 10 ms
(c) 11 ms (d) 23 ms

**7.** Which device splits broadcast domains?
(a) hub (b) router
(c) switch (d) repeater

**8.** An ATM cell is:
(a) 48 bytes (b) 64 bytes
(c) 1500 bytes (d) 53 bytes

**9.** 4000-byte datagram (20 B header), MTU 1500: offset of the last fragment?
(a) 370 (b) 2960
(c) 185 (d) 3

**10.** Hop-by-hop error control is a feature of:
(a) Frame Relay (b) X.25
(c) ATM (d) MPLS

Answer key → 1-c · 2-a · 3-d · 4-a · 5-b · 6-c · 7-b · 8-d · 9-a · 10-b

**Written question:** Explain the count-to-infinity problem of distance vector routing with an example, and describe two techniques that reduce it. 8 marks

1. DV routers know only neighbours' distances (Bellman-Ford), so they cannot tell whether a neighbour's route passes back through themselves 1
2. Chain A–B–C, costs 1. B: A = 1; C: A = 2 (via B). Link A–B fails 1
3. B hears C's "A = 2", sets A = 3 via C; C then sets A = 4 via B; they count up 5, 6, … to ∞ (16 in RIP), with a routing loop the whole time 3
4. **Split horizon**: C does not advertise A back to B, the line it learned it from; **poison reverse**: it advertises A = ∞ to B 2
5. Also hold-down timers and triggered updates; none fully solves loops of 3+ routers, which is why link state is preferred 1

**Before the exam ✓:** Dijkstra on a 5-node graph as a table · a DV update with Bellman-Ford · S = C/(M − ρ) and C + ρt · fragment sizes, offsets, MF · also write: DV vs LS, leaky vs token bucket, X.25 vs FR vs ATM